Cheat sheet

SQL

Queries, joins, windows, performance and the classic interview problems

Query skeleton and order

```sql
SELECT col, AGG(x) AS a        -- 5
FROM t JOIN u ON ...           -- 1
WHERE row_filter               -- 2
GROUP BY col                   -- 3
HAVING group_filter            -- 4
ORDER BY a DESC                -- 6
LIMIT 10;                      -- 7
```

Filtering

Comparison	`=`, `<>`, `>`, `BETWEEN a AND b`, `IN (...)`, `LIKE 'A%'`, `ILIKE` (Postgres)
NULL	`IS NULL`, `IS NOT NULL`, `COALESCE(x, 0)`, `NULLIF(a, b)`
Conditional	`CASE WHEN x > 0 THEN 'pos' ELSE 'neg' END`
Dedupe	`SELECT DISTINCT`, `COUNT(DISTINCT col)`

Joins

INNER	Matches only
LEFT	All left rows; anti-join with `WHERE right.id IS NULL`
FULL OUTER	All rows both sides
CROSS	Cartesian product
SELF	`employees e JOIN employees m ON e.manager_id = m.id`
Semi-join	`WHERE EXISTS (SELECT 1 FROM ...)`

Window functions

```sql
ROW_NUMBER() OVER (PARTITION BY g ORDER BY x DESC)
RANK() / DENSE_RANK() OVER (ORDER BY score DESC)
SUM(x) OVER (ORDER BY d ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
AVG(x) OVER (ORDER BY d ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)   -- 7-day moving
average
LAG(x, 1) OVER (ORDER BY d), LEAD(x) OVER (...), NTILE(4) OVER (...)
FIRST_VALUE(x) OVER (PARTITION BY g ORDER BY d)
```

Classic problems

```sql
-- Nth highest
WITH r AS (SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) rk FROM emp)
SELECT DISTINCT salary FROM r WHERE rk = 3;
-- Top 2 per department
SELECT * FROM (SELECT *, ROW_NUMBER() OVER (PARTITION BY dept ORDER BY salary DESC) rn
FROM emp) t
WHERE rn ⩽ 2;
-- Duplicates
SELECT email, COUNT(*) FROM users GROUP BY email HAVING COUNT(*) > 1;
-- Month-on-month growth
WITH m AS (SELECT DATE_TRUNC('month', d) mon, SUM(amt) rev FROM sales GROUP BY 1)
SELECT mon, rev, ROUND(100.0 * (rev - LAG(rev) OVER (ORDER BY mon)) / LAG(rev) OVER
(ORDER BY mon), 1) pct FROM m;
-- Streaks (consecutive login days)
SELECT user_id, MIN(d), MAX(d), COUNT(*) days FROM
 (SELECT user_id, d, d - (ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY d)) * INTERVAL
'1 day' grp FROM logins) t
GROUP BY user_id, grp HAVING COUNT(*) ⩾ 3;
```

Design and performance

Keys	Primary key (unique, not null), foreign key (referential integrity), unique, check, default
Normal forms	1NF atomic values; 2NF no partial dependency; 3NF no transitive dependency
Indexes	B-tree for equality/range; composite index follows leftmost-prefix rule
EXPLAIN	`EXPLAIN ANALYZE` shows the plan: look for sequential scans on big tables

ACID	Atomicity, Consistency, Isolation, Durability
Isolation levels	Read uncommitted < read committed < repeatable read < serializable
OLTP vs OLAP	Many small transactions vs large analytical scans (warehouses, columnar)
Star schema	Fact table (events, measures) + dimension tables (who, what, when, where)
pgvector	`ORDER BY embedding ⟺ query_vec LIMIT k` (cosine distance)

⚠ **Watch out**

Traps: `= NULL` never matches; `COUNT(col)` skips NULLs but `COUNT(*)` doesn't; integer division in some engines; aliases are not usable in WHERE; LEFT JOIN turns into INNER when you filter the right table in WHERE.